HBC-2 Monitor
Instruction Manual



Summary

I. Color palette

II. The screen

    1. Characters

    2. Cursor, wrap and scroll

III. Native commands

    1. Text

    2. Graphics

    3. Time and VSYNC

    4. Sending commands

IV. The old monitor ports

Color palette

The screen uses a 4-bit color palette, therefore **16 colors** are available. Every command takes a color in the low 4 bits of a byte (the high 4 bits are ignored). At power-on, text is **bright yellow (14) on blue (1)**.

Available colors

Number	Hex	Color	RGB
0	0x0	Black	#000000
1	0x1	Blue	#0000AA
2	0x2	Green	#00AA00
3	0x3	Cyan	#00AAAA
4	0x4	Red	#AA0000
5	0x5	Magenta	#AA00AA
6	0x6	Brown	#AA5500
7	0x7	Light gray	#AAAAAA
8	0x8	Bright black	#555555
9	0x9	Bright blue	#5555FF
10	0xA	Bright green	#55FF55
11	0xB	Bright cyan	#55FFFF
12	0xC	Bright red	#FF5555
13	0xD	Bright magenta	#FF55FF
14	0xE	Bright yellow	#FFFF55
15	0xF	White	#FFFFFF

The screen

The screen is drawn by the **I/O processor** (the ESP32 behind the mailbox, with the FabGL library). It works as a small video card: the CPU sends it short commands, and the ESP32 does the drawing.

Characteristics

	Native display
Resolution	256 × 192 pixels, every coordinate on 1 byte
Colors	16 (see the palette)
Text	8 × 8 pixel characters: **32 columns × 24 rows** Cell (x, y) is at pixel (8x, 8y)
Characters	Windows-1252, codes 0x20 – 0xFF (ASCII and the French accents)
Text colors at start	Bright yellow on blue
Frame rate	60 Hz

There is **one picture**. Text and graphics both draw on it: a character is 8 × 8 pixels of the picture, and graphics can draw over text. The ESP32 also keeps one **text cell** per character position (the character and its colors), so that a program can read a character back and scroll the text. Only the text commands change the cells: a pixel drawn over a character doesn't.

Programs reach the screen in two ways, which draw the same picture:

- the **native commands** (chapter III): a few bytes sent through the mailbox, the fast way;
- the **old monitor ports** (chapter IV): 5 ports written with OUT, kept so that older programs still run.

Characters

The character set is **Windows-1252**:

- 0x20 – 0x7E: ASCII.
- 0xA0 – 0xFF: Latin-1 (é = 0xE9, à = 0xE0, ç = 0xE7, « = 0xAB, ° = 0xB0…).
- 0x80 – 0x9F: the Windows-1252 additions, among them the French ones Latin-1 lacks: € 0x80, … 0x85, Œ 0x8C, ’ 0x92, “ 0x93, ” 0x94, – 0x96, — 0x97, œ 0x9C, Ÿ 0x9F.
- 0x7F and the five codes Windows-1252 leaves undefined (0x81, 0x8D, 0x8F, 0x90, 0x9D) are blank (shaded below).
- 0x00 – 0x1F are control codes (see the table below).

The default font (row = high digit, column = low digit)



Every code from 0x20 to 0xFF can be redefined with **DEFCHAR**. Power-on, the reset button and TEXT_RESET bring back the default font (the public domain font8x8 by Daniel Hepper, plus the Windows-1252 glyphs drawn for the HBC-2).

Source files are UTF-8. The assembler and the C compiler convert string and character literals to Windows-1252: "Noël" gives the bytes 4E 6F EB 6C, 'œ' gives 0x9C. A character that Windows-1252 can't represent is an error.

Control codes (PUTC and PUTS)

Code	Name	What it does
0x08	Backspace \b	Cursor one cell back; from column 0 to column 31 of the row above; nothing at (0, 0). It doesn't erase: send \b, space, \b.
0x09	Tab \t	Cursor to the next column that is a multiple of 8; from column 24 on, to the start of the next row

Code	Name	What it does
0x0A	New line \n	Cursor to column 0 of the next row
0x0D	Return \r	Cursor to column 0 of the same row
Others		Nothing (reserved)

Cursor, wrap and scroll

- The cursor is a text cell, (0, 0) at the top left. PUTC and PUTS draw at the cursor and move it one cell right.
- **Deferred wrap:** a character drawn in column 31 leaves the cursor in column 31 with a **pending wrap**. The next printable character first moves the cursor to column 0 of the next row. \n, \r, \b and CURSOR cancel a pending wrap, so a full row followed by \n moves down one row only.
- **Scroll:** when the cursor must go below row 23, the whole screen moves up one row (8 pixels, graphics included) and the new bottom row is cleared in the text background color. The cursor stays on row 23.
- So writing the bottom-right cell does not scroll: a program can fill the whole screen with text. The screen scrolls only when the next character comes.
- The cursor can be shown as an underline that blinks twice a second (SHOW_CURSOR). It is hidden at power-on.

Native commands

A native command is **one command byte, its arguments**, and a reply only for the commands that read something. The CPU sends them through the mailbox, like the OUT, IN and INFO commands (HBC-2 Instruction Manual, chapter "I/O interface"). All values are bytes.

Each byte takes the CPU about **26 µs** (52 cycles at 2 MHz): one character costs 2 bytes, a full row of 32 characters 34 bytes (0.9 ms), clearing the screen 2 bytes. The ESP32 draws meanwhile; when it is busy, the CPU simply waits before sending the next byte, and nothing is lost.

The native commands need **protocol version 2** (the first byte of the INFO reply). A program must not send them to an I/O processor that doesn't know them: their arguments would be read as commands. Programs find the native display by its descriptor:

Native display descriptor (INFO)

	Native display
Class	03 80 (display, other)
Product ID	0x0006
Ports	1: the port of the VSYNC event

In C, display_port is 0 when there is no native display, and every screen function then returns an error.

Text

Text commands (0x10 – 0x1F)

Code	Name	Arguments	Reply	What it does
0x10	PUTC	c		Draws c at the cursor in the text colors and moves the cursor, or runs the control code c
0x11	PUTS	n, c1 … cn		The same for n characters (0 to 255)
0x12	CURSOR	x, y		Cursor to column x (0 – 31), row y (0 – 23); cancels a pending wrap. Off the grid: ignored

Code	Name	Arguments	Reply	What it does
0x13	COLOR	fg, bg		Text colors of the next characters: fg for the character, bg for its background (and for the rows SCROLL clears)
0x14	CLEAR	bg		Fills the screen with bg; every text cell becomes a space; cursor at (0, 0). COLOR doesn't change
0x15	SCROLL	n		Moves the screen up n rows (graphics too); the rows at the bottom are cleared in the text background. n ≥ 24 clears every row
0x16	DEFCHAR	code, 8 rows		New glyph for code (0x20 – 0xFF): top row first, bit 7 the leftmost pixel. Characters already drawn don't change
0x17	GET_CHAR	x, y	char, colors	The character of cell (x, y) and its colors (bg × 16 + fg). Off the grid: 0, 0
0x18	GET_CURSOR		x, y	The cursor's column and row (column 32 with a pending wrap)
0x19	SHOW_CURSOR	on		1: blinking underline cursor; 0: hidden
0x1A	RECOLOR	which, c		which = 0: the background of every text cell becomes c; 1: the character color. Every cell is drawn again: pixels drawn over the text are replaced
0x1F	TEXT_RESET			Default font and colors, cursor hidden at (0, 0), screen cleared in blue

0x1B – 0x1E are reserved.

Graphics

Graphics commands (0x30 – 0x4F)

Code	Name	Arguments	Reply	What it does
0x30	PIXEL	x, y, c		Pixel (x, y) in color c
0x31	LINE	x0, y0, x1, y1, c		Line, both ends drawn. The same pixels whichever end comes first: drawing it again in the background color erases it
0x32	RECT	x, y, w, h, c		Outline of w × h pixels from (x, y)

Code	Name	Arguments	Reply	What it does
0x33	FILL_RECT	x, y, w, h, c		Filled rectangle of w × h pixels from (x, y)
0x34	GET_PIXEL	x, y	c	The color of pixel (x, y); 0 off the screen

- x is 0 – 255, all on the screen; y is 0 – 191 on the screen, 192 – 255 off it. What falls off the screen is clipped; nothing wraps around.
- **A size of 0 means 256:** FILL_RECT 0 0 0 0 c fills the whole picture, FILL_RECT 0 y 0 8 c a full-width band.
- Graphics don't change the text cells, the cursor or the text colors.
- BLIT (0x40) and the sprites (0x50 – 0x5F) are reserved for later.

Time and VSYNC

Time commands (0x60 – 0x6F)

Code	Name	Arguments	Reply	What it does
0x60	GET_TIME		year, month, day, hour, minute, second	The I/O processor's clock: year since 1900, month 1 – 12, day 1 – 31, hour 0 – 23. Without a clock, six zeros
0x61	VSYNC	on		1: an event every frame (60 Hz); 0: none (at power-on)

The **VSYNC event** comes on the native display's port, with the number of frames (modulo 256) as its data: register I in the handler. At most one VSYNC event waits in the queue: if the CPU hasn't taken it when the next frame comes (interrupts disabled for a while), its count is updated instead of queuing a second one. The handler adds (I − last count) modulo 256 to its tick counter, so no tick is lost. The C library does this for clock() (CLOCKS_PER_SEC = 60) and delay().

SD_READ and SD_WRITE (0x70 – 0x7F) are reserved for the SD card. 0x80 – 0xFF and the reserved codes are ignored, as one-byte NOPs.

Sending commands

- Each command is sent with **interrupts disabled**, so that an interrupt handler's command can never land in the middle of another. A read keeps them disabled until its reply is in.
- The library keeps interrupts disabled for **one command at a time**: between two commands of

the same call, they are allowed again if the caller allowed them. The longest stretch is one PUTS of 255 characters, 6.7 ms. Keys wait in the I/O processor's queue meanwhile; nothing is lost.

- An interrupt handler must not draw: its commands would come between two commands of the call it interrupted, with the cursor and the text colors moved.
- The ESP32 runs the commands in order: a read sees every command sent before it.

Examples (bytes sent)

Bytes	Command	Cost
11 05 48 65 6C 6C 6F	PUTS "Hello"	7 bytes, 0.2 ms
13 0F 01	COLOR white on blue	3 bytes
12 00 17	CURSOR 0, 23	3 bytes
14 01	CLEAR blue	2 bytes (old ports: 0.2 s)
31 00 00 63 00 0C	LINE (0, 0) – (99, 0) bright red	6 bytes, 0.2 ms
60 → 7E 0B 0B 0E 1E 00	GET_TIME: 2026-11-11 14:30:00	7 bytes

Power-on and reset

	Power-on	Reset button	TEXT_RESET
Font	default	default	default
Text colors	yellow on blue	yellow on blue	yellow on blue
Cursor	(0, 0), hidden	(0, 0), hidden	(0, 0), hidden
Screen and text cells	blue, spaces	kept	blue, spaces
VSYNC events	off	off	unchanged
Frame count	0	kept	kept
Old monitor mode	text	text	unchanged

In C (hbc2.h)

Function	Commands sent
screen_putc	CURSOR, COLOR, PUTC, COLOR back
screen_getc	GET_CHAR
screen_write	CURSOR, COLOR, PUTS…, COLOR back
putchar, puts, printf (the terminal)	PUTC, PUTS at the cursor
screen_clear	COLOR, CLEAR, COLOR back
screen_fill	FILL_RECT 0 0 0 0
screen_set_bg, screen_set_txt	RECOLOR 0, RECOLOR 1

Function	Commands sent
screen_scroll, screen_define_char, screen_reset	SCROLL, DEFCHAR, TEXT_RESET
term_set_colors, term_set_cursor_pos, term_get_cursor_pos	COLOR, CURSOR, GET_CURSOR
screen_put_pixel, screen_get_pixel, screen_line, screen_rect, screen_fill_rect	PIXEL, GET_PIXEL, LINE, RECT, FILL_RECT
clock(), delay()	VSYNC
time(), rtc_get	GET_TIME

In assembly, compiler/lib/asm/screen.has has the same routines (screen_putc, screen_write, screen_line…).

The old monitor ports

The monitor's 5 ports are the older, slower way to the same picture, kept so that older programs still run: one character costs 5 OUT commands (15 bytes, about 0.5 ms) instead of 2 bytes. New programs use the native commands.

Monitor descriptor (INFO)

	Monitor
Class	03 00 (display, VGA)
Product ID	0x0001
Ports	5
Interrupts	none (the VSYNC event is the native display's)

Like every device, its first port is found with INFO: monitor_port in C, monitorPort in the assembly project template. The CPU writes and reads the ports with the mailbox's OUT and IN commands (mbx_out and mbx_in in assembly).

List of available ports

ID	Port	Description
0	DATA_0	Colors. Text mode: the background in the high 4 bits, the character color in the low 4 bits (0x1F = white on blue). Pixel mode: the color (0x0C = bright red)
1	DATA_1	Text mode: the character code. Not used in pixel mode
2	POS_X	Column (0 – 31) or x (0 – 255)
3	POS_Y	Row (0 – 23) or y (0 – 191)
4	CMD	Writing it runs a command

List of available commands

ID	Command	Description
0	NOP	No operation
1	WRITE	Text mode: draws DATA_1 in the colors DATA_0 in cell (POS_X, POS_Y), without moving the cursor or changing the text colors. Pixel mode: sets pixel (POS_X, POS_Y) to DATA_0

ID	Command	Description
2	READ	Text mode: the cell's colors on DATA_0, its character on DATA_1. Pixel mode: the color on DATA_0, 0 on DATA_1
3	SWITCH_TO_PIXEL_MODE	WRITE and READ reach the pixels
4	SWITCH_TO_TEXT_MODE	WRITE and READ reach the text cells

- A command runs each time the CPU writes CMD, with the values the other ports hold at that moment. The ports keep their values, so a program only rewrites the ones that change (for example only POS_X and DATA_1 for the next character of a line).
- There is **one picture**: switching mode changes nothing on the screen, it only chooses what WRITE and READ reach.
- A position off the grid (POS_X > 31 or POS_Y > 23 in text mode, POS_Y > 191 in pixel mode) is ignored by WRITE and reads as 0.
- A control code written with WRITE is stored in its cell and drawn blank.
- The old 42-column text mode and its 6 × 8 font are gone: programs that used columns 32 – 41 lose what they put there.
- At power-on and after the reset button, the ports are in text mode. Any other command value does nothing.

In the IDE

The picture is shown in the **Monitor** view, with the blinking cursor when a program shows it. The **I/O** view logs each native command with its first arguments and its reply. The **Peripherals** menu plugs or unplugs the monitor; the native display comes and goes with it (one screen).

The complete references are Peripherals/Native/Reference.md and Peripherals/Monitor/Reference.md, in the Documentation folder.